WG.48.5: Multiple Declarations

Proposal for the Z Standard, by Wolfgang Grieskamp (wg@cs.tu-berlin.de), November 11, 1998, (v2)

What

Global constants can be declared multiple times in the environment of a section. All declarations must be type-compatible.

Why

Methodological Reasons Z specifications are often *literate* and do not only present the pure formalisation but also documentation. For the purpose of documentation, an approved technique is step-wise refinement:

> **The System** We distinguish input and output types:
>
> $[INPUT, OUTPUT]$
>
> Our system is specified by a relation between sequences of inputs and sequences of outputs:
>
> $| \; system : \operatorname{seq} INPUT \; \boldsymbol{\vartheta} \; \operatorname{seq} OUTPUT$
>
> **The Inputs** The possible inputs are specified by the free type below:
>
> $INPUT ::= add \boldsymbol{\theta} NAME \times DATE \boldsymbol{\iota} \mid query \boldsymbol{\theta} NAME \boldsymbol{\iota}$
>
> **The Outputs** ...

The same technique is desirable to be supported in a refinement process over sections boundaries:

> **The Abstract System**
>
> **section** *AbstractSystem* **parents** *toolkit*
> $[INPUT, OUTPUT]$
> ...
>
> **A Refinement of the Abstract System**
>
> **section** *ConcreteSystem* **parents** *AbstractSystem*
> $INPUT ::= add \boldsymbol{\theta} NAME \times DATE \boldsymbol{\iota} \mid query \boldsymbol{\theta} NAME \boldsymbol{\iota}$
> $OUTPUT ::= \ldots$
> ...

One important counter-argument against multiple declarations is that from the point of view of human understanding and of formal proof of specifications it is is desirable to have a unique point of the "defining property" of a Z name. The main argument against: this is only true for *one particular method* how to use Z There are other possible usages of Z:

- It has been suggested that Z may become a formalism for ISO standards . There are many examples (at least before WD 1.5) where properties of single entities are spreaded over various places in the Z standard document itself.
- As reported by Andreas Fett from Daimler-Benz, whose group uses Z mainly for documentation but not for formal proof, they prefer a specification style where they formulate several "views" on the specified system, depending on roles and knowledge-background of readers of the specification, and of the systems components (such as interfaces, functionality, safety-requirements, etc). The views should type-check as individual documents, as well as if they are included in an overall document.
- In ESPRESS, we have combined Z with other formalisms such as statecharts. The combination is founded (and tool-technically supported) by mapping the model provided by Statemate statecharts, inclusive of Statemates data declarations, to Z, and combining this with a Z specification. It is inherent that this can result in multiple declarations, since the user can specify identical entities in the Statemate model and the Z specification.

Since it is hard to predict all the possible usages of Z, the Standard should not restrict the notation because of methodological considerations. Methods, perhaps with accompanying tool-support, which put conditions on the usage of Z going further then the Standard, can be build on top of it.

Technical Reasons Multiple declarations improve orthogonality of Standard Z:

- Multiple declarations are already present on the level of the schema calculus where declarations of the same name are matched under the condition of type-compatibility. Though there is a difference because of the presence of generics on the top-level not available in schemata, the principle concept of multiple declarations is already present in the language.
- It is possible to write e.g.

  $x : \boldsymbol{\upsilon}$ $\qquad$ $x \in \boldsymbol{\gamma}$

  But it is not possible to write the equivalent:

  $x : \boldsymbol{\upsilon}$ $\qquad$ $x : \boldsymbol{\gamma}$

- Whereas for constants arbitrary constraints can be added by axioms (though not by declarations), adding to given types the axiom that they are freely generated is not possible at all (see also proposal WG.48.4).

How

Dependencies This proposal depends on WG.48.4.

Conformance Tools working in batch mode which don't implement this feature may be still regarded as conforming, since syntactic transformations can build which eliminate multiple declarations (see the *ANF* below).

Semantics From any section, one can syntactically build what I would call the *algebraic normal form*, $ANF_S = (G_S, C_S, P_S)$ (see proposal WG.48.3 for details). From ANF_S one can re-construct a Z section $S' = \mathsf{section}\ i\ \mathsf{parents}\ i_1, \ldots, i_n\ \mathsf{END}\ G_S\ C_S\ P_S\ \mathsf{END}$. *Assuming that the semantics equations ensure* that (for each arbitrary arrangement of S') it holds: $\boldsymbol{\delta} S \boldsymbol{\epsilon}^{\boldsymbol{\delta}} = \boldsymbol{\delta} S' \boldsymbol{\epsilon}^{\boldsymbol{\delta}}$, then multiple declarations cause no semantic problems. In particular, it doesn't makes a difference whether a constraint on a variable is put in its declaration or in a property, and whether the declaration precedes a property or not.

Multiple declarations introduce similar problems for the generation of free type axioms as described and solved in WG.48.4, "Semantic Transformations". The reason for this is that one can make a declaration of a given type, then define some other sets in dependency of the given type, and later on add a free-type axiom.

Multiple declarations across section boundaries: Names which are inherited from parents and are redeclared cause no special semantic problems (the *ANF* form can be extended to hierarchies, to explain the semantics). Yet, it is not possible to define mutual recursive free types over section boundaries.

section S **parents** *toolkit*	**section** T **parents** S
$[A];\ B ::= \ldots A \ldots$	$A ::= \ldots B \ldots$

This in general does not result in the meaning one might expect, since the fixing of type B is performed locally in section S for arbitrary A.

Generics Multiple declaration of generic names are allowed provided that the types are equal modulo alpha-conversion.

Operator Templates Multiple declaration of operator templates are allowed provided that they do not change an existing token assignment.

Updates to WD 1.5

- Clause 7.4.4, Lexis:

  > [. . .] If a WORD already has an assignment in the mapping, an operator template can induce the same assignment, but cannot change it [. . .] [Similarly for parents]

- Clause 13.2.2.1 (the according clause in 12, resp.): add a comment such as:

  > Global constants can be declared multiple times in the environment of a section (this environment is build from the parent sections and the paragraphs in the section). All declarations must be type-compatible.

  Change the rule as follows:

  - remove condition $\mathsf{dom}\,\sigma_1 \cap \ldots \cap \mathsf{dom}\,\sigma_n = \{\}$
  - add condition $\forall k, j : 1 \ldots n \bullet \mathsf{dom}\,\sigma_k\ \boldsymbol{\Lambda}\ \sigma_j \sim_{ff} \mathsf{dom}\,\sigma_i\ \boldsymbol{\Lambda}\ \sigma_k$, where $\sim_{ff}$ denotes type equality modulo alpha-conversion of generics for type assignments.

- Clause 15.2.3.1, Semantic Transformation Rules: use the variant for induction described in WG.48.4.

- Clause 16, Semantic Equations: either rewrite this to allow multiple declarations, or introduce the ANF as a semantic transformation to 15. In the second case, multiple declarations across section boundaries need to be still handled, which suggests to use the first version.